DEVSECOPS ACADEMY

Ship what you can defend.

A sixteen-week syllabus in cloud, delivery, and security.

Linux, Git, and GitLab CI. AWS, Terraform, and Ansible. Docker, Kubernetes, and Argo CD. Secret, dependency, static, image, and infrastructure scans. Prometheus, Alertmanager, Grafana, and ELK.

Build nothing you cannot scan, release, and explain.

Graduates leave with a platform they can defend: infrastructure in code, a pipeline that stops when a scan fails, a Kubernetes release, and the signals that say whether it is healthy.

01

Practice

Seventy percent of the hours are labs, projects, and review. Three instructor-led sessions a week, and two to four hours between them.

02

Judgement

A working demo is not enough. Cost, failure, access, and the scan result are graded with the feature.

03

Proof

The capstone is a system another engineer could operate from the runbook, without a private explanation.

WHO IT IS FOR

Beginners with sound computer skills, students, developers, and engineers moving into cloud and security-minded delivery. Basic Linux is useful. A short orientation is provided before Week 1.

A GRADUATE CAN

01 Administer Linux and find a broken service from the logs, the port, and the firewall.

02 Ship through Git and GitLab, and stop a secret before it merges.

03 Build a Docker image, scan it, and refuse to publish what the scan condemns.

04 Provision AWS with least privilege and say what it costs.

05 Write that infrastructure in Terraform and scan the plan before apply.

06 Configure the host with Ansible so a second run changes nothing.

07 Release to Kubernetes through GitLab CI and Argo CD, and roll a bad release back.

08 Page on a real signal, and investigate it in the logs.

The order of the work

The machine and the repository come first. Then the account, the image, the pipeline, the cluster, and the signals. A scan gate sits on that path from the first repository to the capstone.

Month 01 Foundations	01 Linux and the shell	02 Networking and SSH	03 Git and GitLab	04 AWS fundamentals
Month 02 Provision	05 Docker	06 AWS architecture	07 Terraform	08 Ansible
Month 03 Delivery	09 GitLab CI	10 Kubernetes	11 Cluster operations	12 The secure pipeline
Month 04 Operations	13 Metrics and alerts	14 Logging with ELK	15 Argo CD	16 Capstone

SIX GATES. A RED GATE STOPS THE MERGE.

Secrets
Week 3. A credential in the diff never reaches the default branch.

Dependencies
Week 9. A known flaw in a library fails the pipeline.

Static analysis
Week 12. SAST reviews the code on the merge.

Images
Weeks 5 and 12. The container is scanned before the registry keeps it.

Infrastructure
Week 7. Terraform is scanned before apply.

DAST and the record
Week 12. The running app is tested lower down. The build records what shipped.

WEEKS 01 - 04

Foundations

The machine, then the account

01 Linux and the shell

Distributions, the filesystem, and navigation. Users, groups, permissions, ownership, and sudo treated as controls. Packages, processes, systemd, logs, and disk. Bash variables, pipes, grep, find, awk, sed, and scripts.

EVIDENCE A documented server baseline with least privilege, and a Bash report of disk, memory, uptime, and service health.

02 Networking, SSH, and troubleshooting

IP addressing, CIDR, DNS, DHCP, routing, and ports. HTTP, TLS, firewalls, and security groups. SSH keys, scp, rsync, curl, dig, and ss. Patching, and a host baseline of listening ports and failed logins.

EVIDENCE Two Linux hosts with key-based SSH, and a runbook for a deliberately broken web service.

03 Git, GitLab, and secret scanning

Commits, branches, merges, tags, reset, revert, and conflicts. Issues, merge requests, review, and protected branches. READMEs, versioning, and release notes. Secret scanning on the repository, so a credential cannot merge.

EVIDENCE A team project with reviews and tagged releases, plus a planted secret that the scanner blocks.

04 AWS fundamentals

Regions, Availability Zones, and shared responsibility. IAM users, roles, policies, MFA, least privilege, and the CLI. EC2, EBS, S3, VPC, subnets, route tables, and security groups. Budgets, tagging, CloudTrail, and an access review.

EVIDENCE A web server on EC2 inside a restricted VPC, and a static site on S3, with the security and cost decisions written down.

WEEKS 05 - 08

Provision

Image, cloud, code

05 Docker, and the image scan

Containers and virtual machines. Images, registries, layers, and Dockerfiles, including multi-stage builds. Ports, volumes, networks, health checks, and Compose. Resource limits. An image scan, with critical findings treated as defects.

EVIDENCE An application and database in Compose, an optimized image, and a scan report with the serious findings fixed.

06 AWS architecture

Launch templates, Auto Scaling, and an Application Load Balancer. S3 lifecycle, IAM roles for services, Lambda, and events. CloudWatch metrics, logs, and alarms. Public and private subnets, NAT, Secrets Manager, and Systems Manager.

EVIDENCE A highly available sample behind a load balancer, and a Lambda with logs and a CloudWatch alarm.

07 Terraform, scanned before apply

Providers, resources, variables, outputs, and data sources. Init, validate, plan, apply, destroy, and state. Modules, remote state, locking, and separate environments. An infrastructure scan of the plan. Drift, tagging, secrets, and cost.

EVIDENCE A VPC, subnets, security groups, EC2, and S3 in modules. The scan runs before apply. The stack is then destroyed.

08 Ansible

Inventory, modules, facts, and playbooks. Variables, handlers, templates, roles, and Vault. Idempotence, SSH, and privilege escalation. A short hardening pass: updates, SSH, and the firewall. Terraform builds the machine. Ansible configures it.

EVIDENCE A role that installs Nginx and sets a firewall rule, then a second run that changes nothing.

WEEKS 09 - 12

Delivery

Pipeline, cluster, gates

09 GitLab CI, fail closed

Stages, jobs, runners, artifacts, caches, and variables. Rules, environments, and approvals. Build, test, and publish an image. Rolling, blue/green, and canary as ideas, plus a real rollback. Secret and dependency scans fail the pipeline.

EVIDENCE A pipeline that tests and pushes on merge, a planted secret that blocks it, and a demonstrated rollback.

10 Kubernetes

Control plane, nodes, API server, scheduler, and etcd. Pods, Deployments, Services, namespaces, and labels. ConfigMaps, Secrets, probes, requests, and limits. kubectl on kind or Minikube. The first deploy runs as a non-root user.

EVIDENCE A multi-replica application with a Service and probes, then a broken rollout diagnosed from events, logs, and history.

11 Cluster operations

Ingress and network policy. Persistent volumes and stateful workloads. The Horizontal Pod Autoscaler. Helm values, upgrade, and rollback. Security contexts, Pod Security, and RBAC for a restricted account.

EVIDENCE A Helm chart with configurable values, persistent storage, and a service account that can do only its job.

12 The secure pipeline

SAST, dependency scanning, secret scanning, image scanning, and an infrastructure scan, all in GitLab CI. A DAST pass against the test environment. A software bill of materials. Minimal images, pinned versions, no embedded secrets. Findings are fixed or explicitly accepted.

EVIDENCE A planted finding stops the pipeline. The fix is committed. The image runs as non-root. The SBOM is an artifact of the build.

WEEKS 13 - 16

Operations

Signal, then release

13 Prometheus, Grafana, and Alertmanager

Metrics, logs, and traces. Golden signals, and alerts that stay quiet. Prometheus scrape, exporters, labels, PromQL, and alert rules. Alertmanager grouping, routing, inhibition, silences, and receivers. Grafana for the picture. Alertmanager for the page. One route is a security signal: repeated authentication failure.

EVIDENCE A dashboard, and tested routes for CPU, disk, downtime, and login failure, with a short response note.

14 Logging with ELK

Structured logs, levels, and correlation IDs. Elasticsearch indexes. Kibana Discover and dashboards. Ingestion with Elastic Agent, Filebeat, or Logstash. Retention, access, and redaction of secrets that must not sit in the index.

EVIDENCE Application and system logs in Elasticsearch, a Kibana dashboard, and a written investigation of an error or an authentication failure.

15 GitOps with Argo CD

Desired state, reconciliation, and drift. Applications, sync policy, and health. Source kept apart from deployment manifests. Promotion, rollback, and a secrets approach. GitOps does not bypass the scan gates. A manifest still comes from a pipeline that passed.

EVIDENCE A release deployed from Git, a drift correction, a promotion, and a rollback.

16 Capstone and demonstration

Architecture, reliability, security, cost, and the writing, judged together. An incident exercise. A handover. The review asks how a bad release is undone, which signal would page someone, and what the last scan refused to ship.

EVIDENCE The platform on the next page, demonstrated live.

FINAL EVIDENCE

The capstone

A task manager, an inventory service, a student portal, or an API. Individual or a small team. The diagram and the running system describe the same architecture.

01	**Source**	GitLab, with a branching model and merge requests.
02	**Cloud**	Terraform on AWS. Least privilege, tags, and a cost note. Ansible where a host must be configured.
03	**Pipeline**	GitLab CI that tests, builds, scans, and publishes. A failed gate blocks the merge.
04	**Cluster**	Kubernetes with probes, requests and limits, and RBAC. EKS where the budget allows. kind or Minikube otherwise.
05	**Release**	Argo CD from Git, with promotion and rollback shown.
06	**Signals**	Prometheus, Grafana, and Alertmanager routes that have fired. Logs in ELK, used for one investigation.
07	**Gates**	Secrets, dependencies, SAST, the image, and the Terraform plan. A planted finding, and the fix in history. An SBOM from the build.
08	**Record**	Architecture diagram, README, findings, runbook, and a live demonstration.

THE TOOLS, NAMED ONCE

GitLab CI is the pipeline. Secret detection may be GitLab's or gitleaks. Images and Terraform may be scanned with Trivy or Checkov. SAST may be GitLab or Semgrep. The DAST exercise may use OWASP ZAP against the test environment. The point is the gate, not the logo.

What is graded

A claim without a repository, a diagram, or a demonstration does not count.

ASSESSMENT	WEIGHT	WHAT IS JUDGED
Weekly labs	25%	The work of the week, including the troubleshooting.
Projects and review	20%	Deliverables, the writing, and the hygiene of the Git history.
Mid-programme practical	15%	Docker, AWS, Terraform, Ansible, and a pipeline that scans.
Security and signals	15%	Gates that fired, a fix, dashboards, logs, and alerts.
Capstone	25%	Architecture, automation, security, cost, and the demonstration.

THE SHAPE OF A WEEK

Session 1
The idea

The concept, a worked demonstration, and the decision the lab will force.

Session 2
The lab

Guided practice, with checkpoints where the system is expected to break.

Session 3
The review

Project work, someone else's change, a short check, and a presentation.

Between
The hours

Two to four hours for practice, writing, and the portfolio.

TO FINISH

Complete at least 80 percent of the labs. Submit the projects and pass the mid-programme practical. Demonstrate the capstone. In the review, explain the security decisions, the recovery from a bad release, the signal that would page someone, and the cost.

How the lab is run

Accounts, machines, and secrets are handled the way a serious team handles them. Cost control is part of the work.

PRACTICE

01 16 GB of memory is the recommendation once local Kubernetes begins. 8 GB is the floor.

02 Linux by virtual machine, WSL2, or a dedicated machine. GitLab and a container registry are provided.

03 AWS learner accounts use MFA, a budget alert, least privilege, and teardown at the end of every lab.

04 Kubernetes exercises run on kind or Minikube. EKS is reserved for when the budget allows.

05 Credentials never go into Git. A scanner is there to catch the mistake, not to excuse it.

06 Every project leaves a repository, a README, a diagram where the system has one, and a short demonstration.

THE STACK

SYSTEMS
Linux, Bash, SSH, and the network tools.

COLLABORATION
Git and GitLab.

CLOUD
IAM, VPC, EC2, S3, Lambda, CloudWatch, load balancing. EKS as a concept.

AUTOMATION
Terraform and Ansible.

DELIVERY
Docker, GitLab CI, Kubernetes, Helm, Argo CD.

GATES
Secrets, dependencies, SAST, image scan, IaC scan, DAST, SBOM.

SIGNALS
Prometheus, Alertmanager, Grafana, and ELK.